VC1

從一句話到第一個能跑的東西

VC1 — From an Idea to Something That Runs

這一週的難處不在「叫 AI 寫程式」，而在「你到底要什麼」。
寫得清楚，它一次就中；寫得模糊，它會很有自信地做出你不要的東西。

Vibe Coding 專班 · 第 1 週 · 混合程度班 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一份可執行的規格

- 做什麼／給誰用／輸入／輸出／技術限制，五行
- 加上至少 3 條「可測」的驗收條件
- 這份規格 VC3 會拿來寫測試
- 期末專題五項交付物的第一項
- 而且它只花你五分鐘

2 第一個跑得起來的畫面

- 選 CMS 等級與身分別，按一下出現金額
- 不必寫任何一行程式碼
- A 軌用瀏覽器工具、 B 軌用本機工具，都做得到
- 這就是 v0.1 ，之後五週都在改它
- 它會有錯——那是刻意的，第 2 週處理

3 判斷「強度」的能力

- 五級光譜：從行內補全到全自動代理
- 專業的用法是依任務風險調強度
- 不是永遠全開，也不是永遠不用
- 你會知道自己這次落在哪一級、為什麼
- 這個判斷力是六週的主軸

第 1 項是本週真正的主角。多數人會急著叫 AI 動手，然後花三倍的時間來回修正——先花五分鐘寫清楚要什麼，是這門課投資報酬率最高的一件事。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–10	PART 01　Vibe Coding 是什麼	讀。定義、演進、五級光譜	20 分
11–17	PART 02　工具與雙軌	讀 + 決定自己走 A 軌還是 B 軌（含主流工具比較）	20 分
18–26	PART 03　需求規格	本週核心。讀完直接就能動手寫	30 分
27–38	PART 04　動手做（Step 1–8）	開帳號、寫規格、下第一個指令、跑起來	70–100 分
39–40	卡住了？照症狀查	不必從頭讀，卡住時再翻	—
41	常見的四個誤解	做完之後回頭看一次	5 分
42–44	自我檢核、本週交付與回顧	交作業前一定要看	10 分

給 A 軌（零基礎）的提醒：開始之前請先看完 VC0 補強講義（18 頁，約 20 分鐘）。它會說明「網頁是什麼」「瀏覽器在做什麼」「什麼是部署」——這些名詞後面會一直出現。

PART 01

Vibe Coding 是什麼

What It Actually Is

這個詞在 2025 年 2 月才出現，而且從一開始就被誤讀。
它不是「不用懂程式」，是「不必從語法層次開始想問題」。

起源： Karpathy 的那段話，以及它被誤讀的地方

原文（Karpathy， 2025 年 2 月）

- 「有一種新的寫程式方式，我稱為 vibe coding——
  完全交給感覺，擁抱指數成長，忘記程式碼的存在。」

- 他描述的是一種「快速做玩具專案」的體驗：
- 看到什麼就說什麼，不看程式碼，跑不動就再說一次。

- 關鍵是：他講的是「拋棄式的週末專案」，
- 不是「可以交付給別人使用的系統」。

後來被誤讀成什麼

- 誤讀一：「不用懂程式也能開發軟體」
- → 做得出來 ≠ 做得對。這門課六週都在處理這個落差。

- 誤讀二：「這是取代工程師的技術」
- → 對工程師是加速工具；對你是「原本做不到的事現在做得到」。

- 誤讀三：「全自動，不必看它做了什麼」
- → 那是玩具情境。有人會用你的東西，就得有人看得懂、負得起責。

本課程採用的定義：以自然語言描述需求，由 AI 生成程式碼；開發者的角色從「逐行撰寫」轉為「描述意圖、審查結果、引導修正」。重點不是「不用懂程式」，而是「不必從語法層次開始想問題」——你要處理的仍然是邏輯與判斷。

軟體開發的抽象層級演進：這不是斷裂式的革命

年代	階段	人要懂什麼	抽象層級往上一階
1960s–70s	結構化程式設計	電腦怎麼運作（記憶體、暫存器）	從機器指令 → 邏輯結構（ if 、 for ）
1980s–90s	物件導向	怎麼把現實世界拆成物件與關係	從邏輯結構 → 概念模型
2000s–10s	框架與函式庫	怎麼組合現成的元件	從自己寫 → 挑別人寫好的來用
2010s–20s	低程式碼 / 無程式碼	怎麼在介面上把流程拖出來	從寫程式 → 設定流程
2025–	Vibe Coding	怎麼把需求說清楚、怎麼判斷結果對不對	從設定流程 → 描述意圖

這一頁想說的是：每一次抽象層級往上，「要懂的東西」就換一批，但從來沒有變成「什麼都不用懂」。這一次換掉的是語法，換上來的是「說清楚需求」與「判斷結果」——而後兩者恰好是你的專業本來就在做的事。

五級光譜：不是全有或全無

Level 4　全自動代理

給一個目標，它自己規劃、執行、驗證、修正

・風險最高

Level 3　代理式修改

描述需求，它跨檔案改，你審查結果

・本課程主力

Level 2　區塊生成

描述一個功能，它產出一整段，你貼回去用

・對話生成

Level 1　行內補全

AI 補完當前這一行；你仍主導每一個決定

・ Copilot 式

Level 0　手工撰寫

完全自己寫， AI 只用來查文件或解釋錯誤

・最慢，但完全掌握

專業的用法：依風險調強度

低風險、你不熟的領域可放到 Level 3–4 ；高風險、你驗不出對錯的部分降到 Level 0–1

這門課會用到哪幾級

主力在 Level 2–3 。第 6 週的選修 Agent 模組會碰到 Level 4

最危險的組合

「你不熟」 × 「高風險」 × 「 Level 4 」—— AI 寫得出來、你看不出對錯、而且錯了有人受影響

本週要做的事

做完之後說得出「我這次落在哪一級、為什麼停在那裡」。這是本週自我檢核的題目之一

這一頁是六週的主軸。之後每一次你要決定「要不要讓 AI 放手做」，判斷依據都是這張圖：這個任務錯了會怎樣？我驗得出來嗎？——兩個答案決定你該用哪一級。

Copilot 式 vs Agentic 式：兩種完全不同的互動

比較項目	Copilot 式（Level 1–2）	Agentic 式（Level 3–4）
互動方式	你寫，它補。以游標所在位置為中心	你描述目標，它自己決定要改哪些檔案
你看到的東西	灰色的建議文字，按 Tab 接受	一段「我打算這樣做」的計畫，以及改動清單
誰主導	你。每一個決定都經過你	它。你的角色變成審查者
適合什麼	你已經知道要寫什麼，只是打字慢	你知道要什麼結果，但不知道怎麼寫
對不寫程式的人	幾乎沒用——你根本不知道下一行該是什麼	這才是你要的。本課程走這一條
風險	低。它一次只影響一行	高。它可能一次改十幾個檔案，而你看不懂
這門課的做法	—	用 Agentic，但每次都限制範圍、要求它先解釋

最後一列是本課程的核心操作原則，六週都會重複：用代理式的能力，但用「限制範圍 + 先解釋再動手」把風險壓下來。 VC2 會把這件事變成具體的句型。

對工程師是「快」，對你是「能不能」

	× 對軟體工程師				○ 對健康照護背景的你			
原本做得到嗎	工程師	原本就寫得出來	只是比較慢	價值＝省時間	你	原本完全做不到	只能請人做或放棄	價值＝能不能
最大的風險是	技術債累積	對程式碼失去掌握	但他有能力審查品質	可控	不知道自己不知道什麼	看不出哪裡有問題	沒有能力審查品質	這門課要補的
該補什麼	架構與設計	程式碼品質	重構技巧	工程實務	怎麼驗證	怎麼讀懂產出	怎麼把需求說清楚	這六週的內容

這一頁解釋了為什麼這門課的重點跟坊間的 Vibe Coding 課程不一樣。他們教「怎麼更快地生成」，因為那是工程師的痛點；這門課教「怎麼判斷能不能用」，因為那是你的盲區。

你的工作裡，哪些環節適合 Vibe Code

應用場景	很適合	要小心	本課對應
重複性高、邏輯明確的資料整理	是。規則寫得出來就做得出來	資料本身不能是真實個案	VC5 資料分級
一次性的試算與視覺化小工具	最適合的一類。本課程的專案就是	數字要自己查證，不能用 AI 給的	VC3 驗證
把想法快速做成可展示的原型	是。做壞了重來成本很低	原型不等於可上線，界線要講清楚	VC6 交接
你不熟的技術棧，想先做出東西再學	是。這是 Level 3 的典型用法	你驗不出對錯的部分要降級處理	五級光譜
會影響臨床決策的計算邏輯	否	必須有人能完整讀懂並負責	VC3 、 VC5
處理真實個案資料的系統	否	本課程紅線，一律不做	全課程規範

> 判斷的通則就一句話：如果這段程式碼算錯了，會有人受影響嗎？會的話，就必須有人能完整讀懂並負責——不能停在「AI 寫的、跑起來沒問題」。

PART 02

工具與雙軌

Tools & Two Tracks

工具變化很快，但選擇的準則不會變。

這一節結束時，你要決定自己走 A 軌還是 B 軌。

2026 年的工具生態：兩群完全不同的東西

類型	代表工具	怎麼用	適合誰
瀏覽器對話型	Lovable 、 Bolt.new 、 v0 、 Replit 、 Base44	開網頁、註冊帳號、直接對話。 完全不必安裝任何東西	零基礎。本課程 A 軌走這條。 第一堂課的第 40 分鐘就看得到成果
本機代理型	Claude Code 、 Cursor 、 Windsurf 、 GitHub Copilot 、 Antigravity	裝在自己電腦上，直接改你的專案檔案。 終端機或編輯器介面	有程式或資料科學基礎。 本課程 B 軌走這條
差別在哪	—	前者把很多東西藏起來了（比較簡單， 但你不知道發生了什麼）	後者看得到每一個檔案（比較複雜， 但掌握度高）
本課程怎麼處理	—	兩軌並列寫出操作步驟，交的東西完全相同。 第 4 週 GitHub 與部署起兩軌合流	A 軌第 3 週也要讀程式碼—— 避免變成「只會用某一個平台」

※ 本表與後面兩頁的工具比較整理於 2026 年 9 月。工具的方案與免費額度變動頻繁，且多數已從吃到飽轉向用量計費。本講義不寫死任何價格——請在開課前自行確認當期的免費額度與學生方案。這三頁是獨立的，更新它們不影響其他頁面。

瀏覽器對話型（A 軌）：五個主流工具差在哪

工具	出品	它最擅長什麼	要注意什麼
Lovable	Lovable （瑞典）	一段對話就生出「畫面＋資料庫＋登入」整套。 零基礎做得最快、最完整的一個	安全預設偏鬆（權限、金鑰）。 產出照它自己的一套結構，交接前要先整理
Bolt.new	StackBlitz	瀏覽器裡就是一個完整的開發環境， 改完馬上有可分享的網址	定位是原型。離「能上線」還有距離， 程式結構要自己審過
v0	Vercel	畫面與元件做得最漂亮、最合規範 （含無障礙考量）	只做前端元件，不是整套應用。 它產的是 React—— 本課程刻意不用
Replit	Replit	寫、測、部署、託管一站到底， 還能多人同時改同一個專案	產出有自己的習慣寫法， 要換平台或交給別人時最費工
Base44	Wix （2025 年收購）	做得出來的東西可以直接發佈成手機 App	功能越多、綁得越深。 本課程用不到這一段

這五個都做得出本課程的長照試算器，所以不必比誰比較強——對你真正重要的是最後一欄：做完之後帶不帶得走。這也是為什麼第 4 週要求 A 軌也推一次 GitHub：綁在單一平台的成果，換一個工具就帶不走了。

本機代理型（B 軌）：六個主流工具差在哪

工具	出品	它最擅長什麼	要注意什麼
Claude Code	Anthropic	終端機介面，不綁任何編輯器。 跨檔案的長工作最穩，規則檔用 CLAUDE.md	沒有圖形介面，要習慣打指令
Cursor	Anysphere	VS Code 的分支：一邊補全、一邊能整批改多個檔。 規則檔用 .cursorrules	介面太像編輯器，容易一路按 Tab 接受 而忘了自己在審查
Windsurf	Cognition	同樣是編輯器型，強在大專案裡 快速找到相關的那幾段程式碼	同上。本課程的專案只有兩個檔， 這個優勢用不太到
GitHub Copilot	GitHub （Microsoft）	直接長在 VS Code 裡，有代理模式。 學生方案最容易申請	從「補全下一行」起家， 代理能力視方案而定
Google Antigravity	Google	以代理為中心的 IDE，可以同時跑多個代理， 並逐步看它們的工作紀錄	2025 年 11 月才推出、仍在預覽階段， 介面與額度變動快
OpenAI Codex	OpenAI	CLI ＋ 雲端代理，接在既有的 ChatGPT 方案裡	同上，功能與用量規則仍在調整

> 除了 Claude Code 是終端機介面，其餘五個都是編輯器型、操作邏輯幾乎一樣，挑你順手的就好——不要為了選工具花超過十分鐘。 B 軌真正拉開差距的不是工具，而是你有沒有寫規則檔（CLAUDE.md ／ .cursorrules）把技術限制固定下來；沒有它， AI 很可能自己把整個專案改用框架重寫（VC6 第 20 頁）。

選工具之前，先想清楚三件事

1 你的任務規模

- 改幾行、加一個函式 → 行內補全就夠
- 新增一整個功能、跨多個檔案 → 需要代理型
- 本課程專案的規模：中等
- 兩種都做得完，所以不必糾結
- 重點是「你能不能驗證結果」

2 計費方式與用量

- 多數工具已從吃到飽轉向用量計費
- 代理模式特別耗用量（它會反覆讀檔、試錯）
- 學生方案與教育優惠通常存在，申請前先查
- 建議：先用免費額度把流程跑通
- 確認合用再考慮付費（VC5 會教怎麼估）

3 資料會送到哪裡

- 你貼進去的每一個字都會送到服務商
- 個人版與企業版的資料保留政策不同
- 本課程規範：一律不得貼真實個案資料
- 連去識別化的也建議不要
- VC5 會有一整節講這件事

第 3 點是醫療背景學員最該問、卻最常被忽略的一點。這一週只要記住紅線（不貼真實資料）；第 5 週會完整說明政策差異、什麼算去識別化、以及院內可控部署的選項。

什麼情況用什麼強度：兩個問題

你要決定「這次要不要讓 AI 放手做」

1. 這件事錯了，會有人受影響嗎？
   - 是 → 會 → 降到 Level 0–1，或不要用 Vibe Coding
     例：臨床劑量計算、給付金額的核定邏輯、會寫進病歷的內容
     這一題是紅線題，先問這個
   - 否 ↓

2. 你驗得出對錯嗎？
   - 是 → 驗不出來 → 不要放手。先找到能驗證的方法再說
     「驗得出來」的意思是：你知道正確答案應該是多少，或有人能幫你確認
     本課程的專案驗得出來——因為給付基準是公開的
   - 否 ↓

3. 你熟這個技術嗎？
   - 是 → 不熟 × 低風險 → 最適合放到 Level 3。學習效果也最好
     熟 × 低風險 → 用 Level 1–2 就好，你自己寫可能還比較快
     不熟 × 高風險 → 最危險的象限。找人審查，或不要做
   - 否 ↓

4. 本週的任務落在哪裡
   - → 低風險（教學專案、公開資料、做壞了重來）
     驗得出來（給付基準是公開的，你算得出正確答案）
     → 所以本週可以放心用 Level 3。這也是刻意的安排

注意兩個問題的順序：先問「錯了會怎樣」，再問「我驗不驗得出來」。很多人只問第二題就決定放手，但如果第一題的答案是「有人會受傷」，那再怎麼會驗證都應該更保守。

你走 A 軌還是 B 軌？（現在就決定）

	A 軌　零基礎	B 軌　有程式或資料科學基礎
這樣的人	臨床、長照、行政人員；沒寫過程式； 不想在第一週花兩小時裝環境	修過本校資料科學課程； 用過 Python 、 AI Studio 或寫過任何程式
主要工具	Lovable 或 Bolt.new （瀏覽器，免安裝）	Claude Code 或 Cursor （本機安裝）
開課前要準備	一個瀏覽器帳號即可	Node.js LTS 、 Git 、編輯器、 AI 工具帳號
講義怎麼看	凡是標「A 軌」的段落照做，標「B 軌」的可略過	反之亦然
第 4 週起	兩軌完全合流。 GitHub 、部署、資料、交接一致	同左
可以換嗎	可以。第 3 週結束前隨時能換——	但同一週內請走同一條，不要混著做

選錯了沒關係，但請現在就選一個。最糟的情況是「兩邊都試一下」——那會讓你在兩套操作邏輯之間切換，什麼都學不熟。不確定的話選 A 軌，它的門檻低很多。

PART 03

需求規格

Writing a Spec

本週的核心。 AI 寫程式的能力已經很強，錯誤多半來自「你沒說清楚」。
而說清楚需求，恰好是不寫程式的人最有優勢的地方。

需求規格的三層：由下往上越來越精確

第三層　驗收條件

「怎樣才算做對了」——可以實際測的句子

- 第 5 級低收 → 0
- 第 8 級上限 36,180
- VC3 會變成測試

第二層　使用者故事

誰、在什麼情況下、需要什麼、為了什麼

- 照管專員
- 家訪時
- 快速估算

第一層　五行規格

做什麼／給誰用／輸入／輸出／技術限制

- 五分鐘寫完
- 貼給 AI 的就是這個

為什麼要三層

第一層讓 AI 知道要做什麼；第二層讓你自己想清楚使用情境；第三層讓你（和 AI）知道怎樣算對

最常被跳過的是第三層

沒有驗收條件，「做完了」就變成一種感覺。而感覺是沒辦法交付、也沒辦法評分的

三層要花多久

五行規格 5 分鐘、使用者故事 5 分鐘、驗收條件 10 分鐘。二十分鐘，省下之後好幾小時的來回

期末專題的要求

規格書要含五項齊全 + 至少 3 條可測的驗收條件。現在寫的這一份就是範本

這一頁是本週的骨架，後面七頁都在拆解它。實務上你不必每次都寫完整三層——但「五行規格 + 驗收條件」這兩層，只要是會給別人用的東西就一定要有。

五行規格：五個欄位，各寫什麼

欄位	要回答的問題	寫得好的樣子	常見的錯
① **做什麼**	這是一個什麼東西?	一個單頁的長照給付試算器	「一個長照系統」——太大， AI 會亂猜
② **給誰用**	誰會打開它？在什麼情況下?	照管專員與家屬。手機也要能正常顯示	沒寫。 AI 會預設是給工程師看的
③ **輸入什麼**	使用者要提供哪些資訊？型態是什麼?	CMS 等級： 2–8 的下拉選單； 身分別：一般／中低收／低收	只寫「等級」，沒寫範圍與型態
④ **輸出什麼**	他會看到什麼？幾個數字?	每月給付上限、部分負擔比率、 民眾自負額、政府補助，四項	「算出金額」——哪一種金額?
⑤ **技術限制**	不可以做什麼？必須怎麼做?	單一 HTML 檔 + 一個 calculator.js ； 不要用任何框架；資料寫死在程式裡	沒寫。這一欄漏掉的代價最高

第 ⑤ 欄最容易被忽略，卻最重要——下一節會單獨講。簡單說：不寫技術限制， AI 會自己挑一套它最熟的技術棧，可能是你完全裝不起來、也看不懂的東西。

兩個完整的五行規格範例（照這個深度寫）

範例一：本課程的貫穿專案

【要做什麼】
一個單頁的長照給付試算器

【給誰用】
照管專員與家屬。
手機也要能正常顯示（家屬多半用手機）

【輸入什麼】
・CMS 等級：2–8 的下拉選單
・身分別：一般戶 / 中低收 / 低收

【輸出什麼】
・每月給付上限（新臺幣）
・部分負擔比率
・民眾自負額
・政府補助金額

【技術限制】
單一 HTML 檔＋一個 calculator.js。
不要用任何框架。額度表直接寫在程式裡。

1 每一項都具體到可以照做

「2–8 的下拉選單」比「等級」精確得多
精確到 AI 不必猜的程度

2 情境寫在「給誰用」裡

「家屬多半用手機」這一句，
直接決定了它會不會做手機版

3 輸出列成清單

四項，不多不少
之後驗收就照這四項一個一個看

4 技術限制寫死

「不要用任何框架」這一句，
讓 A 軌零基礎的人也跑得起來

這份規格總共 15 行，寫完大約五分鐘。它讓 AI 第一次產出就八成接近你要的——而不寫的話，你會花三十分鐘來回解釋「不是這樣」。

五行規格範例二：一個不同領域的例子

範例二：換一個題目也是同樣的寫法

【要做什麼】
一個單頁的「用藥時間提醒表」產生器

【給誰用】
出院衛教時，護理師列印給家屬帶回家。
使用者多為高齡者，字要大、要能直接列印

【輸入什麼】
・藥名（文字）
・一天幾次：1 / 2 / 3 / 4 次的選項
・飯前或飯後：兩個選項
・可新增多筆（最多 10 筆）

【輸出什麼】
・一張表格：時段（早 / 中 / 晚 / 睡前） × 藥名
・一個「列印」按鈕，列印時不要出現網頁選單

【技術限制】
單一 HTML 檔。不要用框架、不要存任何資料。
所有資料只留在瀏覽器，不上傳。

1 情境決定了設計

「高齡者」→ 字要大
「列印給家屬」→ 要有列印按鈕

2 數量限制也要寫

「最多 10 筆」——不寫的話 AI 可能做無限筆
邊界從規格階段就要定

3 輸出寫得夠具體

「時段 × 藥名 的表格」
而不是「產生提醒表」

4 隱私也是技術限制

「不存任何資料、不上傳」
醫療場域的規格一定要有這一行

兩個範例的共同結構完全一樣。你自己的題目也照這五個欄位填——特別注意「給誰用」那一欄，它常常隱含了最重要的設計決定（手機、字大小、列印、隱私）。

驗收條件：把「好用」變成「測得出來」

不可測

不可測的寫法
「算得正確」
「介面要好用」
「手機也能看」
「要處理錯誤」
「速度要快」



可測

可測的寫法
第 5 級 × 低收 → 自負額顯示 0
三次點擊之內完成一次試算
寬度 390px 下不需左右拖曳
未選身分別按計算 → 出現提示文字
按下計算後 1 秒內顯示結果

判斷標準只有一個　「另一個人拿著這句話，能不能自己判斷通過或沒通過？」能，就是可測的

要有具體的數字　「第 5 級」「390px」「1 秒」——沒有數字的條件多半不可測

正確答案從哪來　從你的專業知識或官方文件查來，不是 AI 給的。這一點 VC3 會再三強調

寫幾條　本週要求至少 3 條。期末專題也是 3 條起跳——挑「錯了最嚴重」的那幾件事來寫

這一頁是本週最實用的一頁。你交的規格會不會被退回，就看驗收條件寫得可不可測——而這件事跟程式完全無關，純粹是「把話說清楚」的能力。

一條驗收條件，要從四個角度想

你想到的那一句話

「低收入戶不論等級，自負額都是 0」

先寫下這一句

從四個角度展開

寫成驗收條件

① 正常情況 — 最典型的那一種

- 第 5 級 × 低收入戶
- 自負額應為 0

→ 最基本的一條

② 邊界情況 — 最小與最大

- 第 2 級 × 低收 → 0
- 第 8 級 × 低收 → 0

→ 錯誤最愛藏這裡

③ 對照組 — 不該被影響的

- 第 5 級 × 一般戶 → 3,856
- 改對低收不能影響它

→ 防「改 A 壞 B」

④ 例外情況 — 使用者做錯事的時候

- 不選身分別按計算
- 應出現提示，不是沒反應

→ 常被忘記

這四個角度 VC3 會再用一次——到時候是把它們變成 test.html 裡的測試案例。所以現在寫得越具體，第 3 週越省事。第 ④ 個角度最常被忘記，而它恰好是使用者最先撞到的。

範圍蔓延：怎麼擋住「順便再加一個」



實用做法：另外開一張「以後再說」的清單。想到什麼就記上去，但這一輪絕對不做——這樣你既不會忘記那個好點子，也不會讓現在這一輪永遠做不完。期末專題也建議這樣管理。

為什麼「技術限制」那一欄最重要

不寫的話，AI 會自己挑一套技術棧

- 它多半會挑目前最流行的框架——那對它來說是「最好的做法」
- 但對你來說可能是：裝不起來、跑不動、看不懂、部署不出去
- 而且它不會問你，它會直接做，然後你要花兩小時才發現裝不起來

本課程建議寫死的三句話

- 「單一 HTML 檔 + 一個 calculator.js」——最單純，A 軌也跑得動
- 「不要使用任何框架或套件」——不必安裝任何東西
- 「所有資料只留在瀏覽器，不要上傳」——醫療場域的隱私底線

這一欄還可以寫什麼

- 不可以做什麼：「不要加上使用者登入」「不要接任何外部 API」
- 必須怎麼做：「計算邏輯放在單獨的檔案裡」（這一句讓 VC3 的測試寫得出來）
- 環境限制：「我用的是 Windows」「不要用需要安裝的工具」

注意第二點的第三句：「計算邏輯放在單獨的檔案裡」。這一句在第 1 週看起來沒什麼意義，但它決定了第 3 週能不能寫測試——好的技術限制是有遠見的。

PART 04

動手做： Step 1–8

Hands-on

從這裡開始請打開你的工具，跟著做。 Step 1 兩軌並列，各看自己那一頁。
目標：一小時後，你有一個能算出金額的畫面。

Step 1　A 軌　開帳號並開啟第一個專案

你現在要做什麼　▶　B 軌請跳過這一頁，看下一頁

① 註冊

到 Lovable 或 Bolt.new 的網站，用 Google 帳號登入即可。
不必安裝任何東西，不必付費。

② 開一個新專案

首頁會有一個大的輸入框（「你想做什麼？」）。
先不要打字——下一步才會用到它。
有些工具會先問你要不要選範本，選「空白」或直接略過。

③ 認識畫面（大概看一下就好）

左邊：跟 AI 對話的地方
右邊：預覽（你做出來的東西長什麼樣）
通常還有一個「Code」或「檔案」分頁——現在不用管

你應該看到什麼

你會看到一個左右分割的畫面：
左邊可以打字跟 AI 說話，
右邊是空白的預覽區。

這樣就準備好了。
先不要開始做，回頭把規格寫完（Step 2）。

沒看到的話，先檢查這幾件事

① 要求付費才能建立專案
→ 換一個工具試。多數都有免費額度
② 介面跟講義寫的不一樣
→ 工具改版很快。找「對話框」與「預覽」這兩個區塊就對了
③ 不確定該選哪一個工具 → 兩個都試五分鐘

A 軌的優勢就在這一頁：五分鐘之內你就準備好了，不必裝任何東西。代價是很多細節被藏起來——所以第 3 週會要求你也讀一次程式碼，第 4 週兩軌會合流。

Step 1B　B 軌　安裝工具並開啟專案資料夾

你現在要做什麼 ▶ A 軌請跳過這一頁

① 準備環境（開課前應已完成）

・Node.js LTS　・Git　・VS Code 或同等編輯器
這三樣裝好之後，本課程六週都不必再裝別的

② 裝一個 AI 編碼工具，二選一

Claude Code：終端機介面，跨編輯器
Cursor：VS Code 的分支，圖形介面
兩個都可以，選你順手的

③ 建立專案資料夾

新建一個空資料夾，命名 ltc-calculator
路徑不要有空格與中文（避免之後的麻煩）
用你的工具開啟這個資料夾

你應該看到什麼

工具能看到你的資料夾（目前是空的），
而且對話框可以打字。

B 軌的進階挑戰（不計分）：
在資料夾裡建一個 CLAUDE.md 或 .cursorrules，
把技術限制寫進去當成專案常規。

沒看到的話，先檢查這幾件事

① 路徑有中文或空格
　→ 現在改還來得及。之後部署會麻煩
② 工具看不到資料夾
　→ 確認是「開啟資料夾」不是「開啟檔案」
③ 額度用完了
　→ 換另一個工具，或改走 A 軌

給 B 軌的提醒：你的優勢是看得到每一個檔案，但也因此更容易一開始就想太多（要不要用框架？要不要拆模組？）。這一週的目標是「跑得起來」——複雜度留到真的需要時再加。

Step 2　寫下你的五行規格

你現在要做什麼　▶　先不要碰工具。拿一張紙或開一個空白文件，把五個欄位填完

空白樣板（直接抄）

【要做什麼】
【給誰用】

【輸入什麼】
【輸出什麼】

【技術限制】

三個提醒

① 「給誰用」要寫使用情境（在哪裡用？用什麼裝置？）
② 「輸入」「輸出」要具體到型態與數量
③ 「技術限制」至少寫三句（見第 26　頁）

你應該看到什麼

一份 12–15 行的規格，五個欄位齊全。

判斷寫得夠不夠的方法：
拿給旁邊的人看，
問他「你知道我要做什麼嗎？」
他答得出來，就夠了。

沒看到的話，先檢查這幾件事

① 寫不出「技術限制」
　→ 抄第 21 頁範例一那三句就好
② 覺得寫規格很浪費時間
　→ 五分鐘。而不寫的話，
　　你會花三十分鐘來回解釋
③ 想直接照抄範例 → 可以，但至少改「給誰用」

這一步刻意要你「先不要碰工具」。一旦打開對話框，人的本能就是開始描述、開始看它回什麼——那會讓你在還沒想清楚的時候就被它的產出帶著走。

Step 3　把規格轉成 3 條可測的驗收條件

你現在要做什麼　▶　從你的規格挑出「錯了最嚴重」的三件事，各寫成一句可測的話

改寫範例（不可測 → 可測）

「算得正確」
→ 第 5 級 × 低收入戶，自負額顯示 0

「手機也能看」
→ 寬度 390px 下，畫面不需左右拖曳

「要處理錯誤」
→ 未選身分別按計算，畫面出現提示文字

檢查每一條

① 有沒有具體的數字或條件?
② 另一個人能不能自己判斷通過或沒通過?
③ 我知道正確答案是多少嗎? 從哪查來的?

你應該看到什麼

三條句子，每一條都有具體數字。

把它們寫在規格的最後面，
跟五行規格放在同一份文件裡。

這三條 VC3 會直接變成測試——
所以現在寫得越具體，第 3 週越省事。

沒看到的話，先檢查這幾件事

① 想不出三條
　→ 從「輸出什麼」那一欄找。
　　每一項輸出至少可以寫一條
② 不確定正確答案是多少
　→ 先去查。查不到就換一條寫。
　　不能用「AI 說的」當標準

第 ③ 個檢查最重要：驗收條件的正確答案必須來自你的專業知識或官方文件。如果標準本身是 AI 給的，那整套驗證只是在驗證它跟自己一致——這是 VC3 會反覆強調的一點。

Step 4　第一個指令怎麼下

你現在要做什麼　▶　把規格貼進對話框，並加上兩句關鍵的話

可以直接照抄（把〔〕換成你的規格）

請幫我做一個〔要做什麼〕，給〔給誰用〕使用。

輸入：
〔逐條列出〕

輸出：
〔逐條列出〕

技術限制：
〔逐條列出〕

動手之前，請先說明你打算怎麼實作、會建立哪些檔案。
我確認之後再請你開始寫。

你應該看到什麼

它會先回你一段「實作計畫」：
打算建幾個檔案、每個檔案做什麼、
畫面大概長什麼樣。

先不要說「開始吧」——
下一頁要教你怎麼讀這段計畫。

沒看到的話，先檢查這幾件事

① 它直接就開始寫了
→ 沒關係，但要它補說明「你建了哪些檔案、各做什麼」
② 它問你一堆問題
→ 好現象，代表規格還有不清楚的地方。一題一題回答，順便補進你的規格

最後兩句「先說明再動手」是整份指令最有價值的部分。它讓你在它寫下第一行程式碼之前，就有機會攔下誤解——比做完再改省事十倍。這兩句話六週都會一直用到。

怎麼讀它回的實作計畫：三個該糾正的地方

AI 回的計畫（實際會像這樣）

我打算這樣實作：

1. 使用 React ＋ Vite 建立專案，
   以元件化的方式組織畫面。

2. 建立以下檔案：
   - src/App.jsx （主畫面）
   - src/components/Calculator.jsx
   - src/data/rates.json （額度資料）

3. 額度資料我會先用範例數值，
   您之後可以自行替換。

4. 另外我會加上深色模式切換與
   結果匯出 PDF 的功能。

需要我開始嗎?

1 技術棧不符合限制

你寫的是「不要用任何框架」，
它卻用了 React ＋ Vite → 要求改成單一 HTML

2 「先用範例數值」＝它要編數字

這是最危險的一句。給付金額不能編
→ 把正確的額度表直接貼給它

3 多做了你沒要求的功能

深色模式、匯出 PDF 都不在規格裡
→ 要求拿掉，記進「以後再說」清單

4 檔案結構可以接受

拆成畫面與計算兩部分是對的
但要改成 index.html ＋ calculator.js

這一頁是本週最值得花時間的一頁。四個問題裡有兩個是紅色的——而它們在做完之後才會發現的話，代價分別是「整份重來」與「金額全錯」。讀計畫只要一分鐘。

Step 4-2　把糾正說回去

你現在要做什麼　▶　照下面的格式，一次把三件事講清楚

可以直接照抄

謝謝，有三個地方要調整：

1. 請不要使用 React 或任何框架。
   改成單一的 index.html ＋一個 calculator.js。

2. 額度數字請不要自行假設。使用以下數值：
   CMS 2–8 級：10020 / 15460 / 18580 / 24100 / 28070 / 32090 / 36180
   部分負擔：一般戶 16%、中低收 5%、低收 0%

3. 請不要加入深色模式與匯出 PDF，
   這一版只做規格裡的四項輸出。

調整後請再說明一次你的計畫。

你應該看到什麼

它會回一份修正後的計畫：
單一 HTML ＋ calculator.js，
用你給的數字，沒有多餘的功能。

這一份看起來對了，
才說「請開始」。

沒看到的話，先檢查這幾件事

① 它又加了別的東西
→ 再糾正一次。這很正常，不必客氣
② 它問「確定不要用框架嗎」
→ 確定。回「是的，請用單一 HTML」
③ 你不確定數字對不對
→ 範例的 README 有完整的額度表

第 2 點的寫法值得學起來：凡是有標準答案的數字（金額、劑量、法規門檻），一律自己查好貼給它，不要讓它「先用範例數值」。這是醫療場域使用 AI 的基本紀律。貼的時候順手記下出處與查證日期——這七個數字出自衛福部照顧及專業服務給付額度，2026 年 9 月查證時長照 3.0 上路後尚未調整。

Step 5　跑起來看看

你現在要做什麼　▶　說「請開始」，等它做完之後打開來看

A 軌

右邊的預覽區會自動更新，直接就看得到。
建議點「在新分頁開啟」，看起來比較完整。

B 軌

它會建立 `index.html` 與 `calculator.js` 兩個檔案。
在檔案總管裡雙擊 `index.html` 就能打開。
（不需要 `npm run dev`— 這是單純的 HTML 檔）

打開之後做這三件事

① 選第 5 級 + 一般戶 → 按計算
② 看有沒有出現四個數字
③ 自己算一次：`24,100 × 16%` = ?

你應該看到什麼

你應該看到：
兩個下拉選單、一個計算按鈕，
按下之後出現四個數字。

第 5 級 × 一般戶的自負額
應該是 3,856 。
對得上就成功了。

沒看到的話，先檢查這幾件事

① 畫面空白
　→ B 軌：確認兩個檔在同一個資料夾
　→ A 軌：等它跑完，或重新整理預覽
② 數字對不上
　→ 先別急著修。第 2 週整週在做這件事
　　這一週只要「跑得起來」就算過

重要提醒：這一週的目標是「有一個能跑的東西」，不是「完全正確」。它很可能有錯——那是刻意的，第 2 週會用整整一週來找出並修好三個 bug 。現在不要提前處理。

Step 6　存檔與命名

你現在要做什麼　▶　把這一版好好存下來，之後五週都在改它

A 軌

工具會自動保存。但請把專案名稱改成有意義的，
例如 `ltc-calculator`，不要留 `untitled-project`。

另外：找到「下載」或「Export」功能，
把整包下載一份到自己電腦，當作備份。

B 軌

確認資料夾裡有 `index.html` 與 `calculator.js`。
把整個資料夾複製一份，命名 `v0.1_` 備份。

（第 4 週會學正規的版本控制，
在那之前先用這個土法）

你應該看到什麼

專案有一個有意義的名字，
而且你手上有一份備份。

為什麼要備份：
第 2 週要請 AI 改東西，
改壞了才有地方可以退。

沒看到的話，先檢查這幾件事

① 找不到下載功能
→ 多半在專案設定或右上角選單裡
② 覺得備份很麻煩
→ 第 4 週學 Git 之後就不必了。
在那之前這是唯一的退路

這個土法（複製一份資料夾）會用到第 4 週。到時候你會學 git commit 與 git revert，那才是正規做法——但先體驗一下「沒有版本控制有多不方便」，學 Git 時會更有感。

Step 7　用你的驗收條件自己驗一次

你現在要做什麼　▶　拿出 Step 3 寫的三條驗收條件，一條一條實際測

怎麼測（以本課程的三條為例）

條件 1　第 5 級 × 低收入戶 → 自負額應顯示 0
實際顯示：_____　　○ / ×

條件 2　寬度 390px 下不需左右拖曳
用開發者工具切手機模式看（VC2 會教）
或先把瀏覽器視窗拉很窄　　○ / ×

條件 3　未選身分別按計算 → 出現提示文字
實際情況：_____　　○ / ×

你應該看到什麼

三條裡很可能有一到兩條是 ×。

這是正常的，也是預期中的——
不要現在修。
把 × 的那幾條記下來，
第 2 週的第一個任務就是處理它們。

沒看到的話，先檢查這幾件事

① 三條全部都是 ○
→ 檢查一下條件是不是寫得太寬鬆。
試試第 8 級、試試不選身分別
② 不知道怎麼測第 2 條
→ 把瀏覽器視窗拉到很窄就大致看得出來

這一步是本週與 VC3 的橋樑：你現在做的「拿條件一條一條對」，就是驗證的最原始形式。第 3 週會把它變成自動化的測試，三秒跑完 29 項——但概念完全一樣。

Step 8　回答一個問題：我這次落在第幾級?

你現在要做什麼　▶　看第 7 頁的五級光譜，說出你這次的強度並寫下理由

要回答的三件事

① 我這次落在 Level　幾?

② 為什麼停在那一級?
（是工具能力? 我的驗證能力? 還是任務風險? ）

③ 如果任務換成「臨床用藥劑量計算」，
我應該調到第幾級? 為什麼?

你應該看到什麼

多數人這一週落在 Level 3：
描述需求、它跨檔案做、你審查結果。

第 ③ 題的答案應該是「降到 0–1 」，
理由是「算錯會有人受影響，
而且我驗不出對錯」。

沒看到的話，先檢查這幾件事

① 不確定自己在第幾級
→ 問自己：我有沒有逐行看過它寫的東西?
沒有 → 至少 Level 3
② 覺得這題很虛
→ 這是本課程的主軸。六週後你會發現
這個判斷比任何技術都重要

這一題會在自我檢核與課後反思再出現一次。六週結束時，能不能依任務風險自覺地調整強度，是這門課最想教會你的一件事——比任何一個工具或指令都重要。

卡住了？照症狀查（一）規格與指令

症狀	可能原因	怎麼處理
AI 做出來的東西跟我想的差很多	規格太籠統，它只好猜	回到第 20 頁，把五個欄位重新填一次。 特別檢查「給誰用」與「技術限制」有沒有寫
它用了我看不懂、裝不起來的技術	技術限制那一欄沒寫， 或寫得不夠明確	明確要求：「請用單一 index.html ＋一個 calculator.js， 不要使用任何框架或需要安裝的套件」
它自己編了數字	你沒有把正確數字給它	凡是有標準答案的數字，一律自己查好貼給它。 看到「先用範例數值」這種說法就要警覺
它加了一堆我沒要求的功能	沒有限制範圍	要求拿掉，並記進「以後再說」清單。 下次在指令裡加「這一版只做規格裡的項目」
它問我一堆問題，我答不出來	好現象——代表規格有 不清楚的地方	一題一題想清楚再答，並把答案補進規格。 這是在幫你把需求想完整
改了三次還是不對	可能是規格本身有矛盾	停下來重讀自己的規格。 或開新對話，把完整規格重貼一次

六個症狀裡有四個的根源都是同一件事：規格沒寫清楚。這也是為什麼本週把整整九頁給了「需求規格」——它不是形式，它是你之後五週的地基。

卡住了？照症狀查（二）工具與畫面

症狀	A 軌怎麼辦	B 軌怎麼辦
畫面一片空白	等它跑完，或重新整理預覽區。 仍空白就問 AI「預覽沒有內容，可能是什麼原因」	確認 index.html 與 calculator.js 在同一個資料夾，檔名大小寫一致
按了計算沒反應	點「在新分頁開啟」再試一次	按 F12 看 Console 有沒有紅字 （VC2 會詳細教）
工具介面 跟講義不一樣	工具改版很快。找「對話框」與 「預覽」這兩個區塊就對了	同左。指令與檔案結構不會變
免費額度用完了	換另一個工具（Lovable / Bolt.new / Replit 都可以），規格直接重貼	換另一個工具，或改走 A 軌
完全不知道 下一步該做什麼	回到第 3 頁的本週地圖，找到你現在在哪一步	同左

最後一列是給自學者的：卡住的時候不要憑感覺往下亂試，回到地圖確認「我在第幾步、下一步是什麼」。每一個 Step 都有「你應該看到什麼」——沒看到就先停下來，不要往下走。

常見的四個誤解（做完之後回頭看一次）

誤解	為什麼是錯的	正確的做法
① 寫規格是浪費時間	不寫的話，時間會花在來回解釋「不是這樣」	五分鐘寫規格，省下三十分鐘的來回。這是這門課投資報酬率最高的一件事
② 「算得正確」就是驗收條件	不可測。另一個人拿著這句話沒辦法判斷通過或沒通過	改成「第 5 級 × 低收 → 自負額顯示 0」。要有具體的數字與條件
③ AI 給的數字應該是對的	它沒有查過任何資料。「先用範例數值」＝ 它在編	凡是有標準答案的數字，一律自己查好貼給它。這是醫療場域的基本紀律
④ 第一版就要完全正確	幾乎不可能，而且不必要	本週目標是「跑得起來」。它有錯是刻意的——第 2 週整週在處理

第 ③ 個誤解在醫療場域的代價最高。 AI 會用非常有自信的語氣給你一個編出來的數字，而且格式完全正確——這種錯誤不會報錯、不會提醒你，只有「知道正確答案」的人抓得到。

本週自我檢核（答不出來請回去看標註的頁數）

Q 1 **五行規格是哪五行？哪一行最常被漏掉、漏掉的代價是什麼？**

做什麼／給誰用／輸入／輸出／技術限制。最常漏第五行 → 第 20 頁與第 26 頁

Q 2 **怎麼判斷一條驗收條件是不是「可測」的？**

另一個人拿著這句話，能不能自己判斷通過或沒通過 → 第 23 頁

Q 3 **AI 回的實作計畫裡，哪兩種說法你看到就要警覺？**

① 用了你限制以外的技術棧　② 「先用範例數值」＝它要編數字 → 第 33 頁

Q 4 **你這次落在五級光譜的第幾級？如果任務換成臨床劑量計算，該調到第幾級？**

這一題沒有標準答案，但你要說得出理由 → 第 7 頁與第 38 頁

第四題是本課程的主軸題，六週都會再出現。能依「錯了會怎樣」與「我驗不驗得出來」兩個問題自覺地調整強度，比記住任何一個指令都重要。

本週交付與驗收條件

交付物	驗收條件（全部達成才算通過）	怎麼查
① **規格書**	五個欄位齊全（做什麼／給誰用／輸入／輸出／技術限制）， 且「技術限制」至少三句	文件檢查
② **驗收條件**	至少 3 條，每一條都有具體數字或條件， 且寫得出「怎麼算通過」	同儕互評—— 對方看得出怎麼測才算過
③ **能跑的畫面**	選 CMS 等級與身分別後出現金額。 不要求完全正確（第 2 週處理）	當場操作一次； 自學者對照 範例 \v0.1
④ **驗證紀錄**	三條驗收條件各測一次，記下 ○ 或 ×	看紀錄。有 × 是正常的
⑤ **一句話**	說得出自己這次落在五級光譜的哪一級、為什麼	自我檢核第 4 題

第 ① 項與第 ② 項是期末專題交付物的第一項，現在寫的就是範本——請留著，期末直接在這個基礎上改寫。第 ③ 項刻意不要求正確：這一週只驗「做得出來」，第 3 週才驗「做得對」。

第 1 週重點回顧

1. 難處不在「叫 AI 寫程式」，而在「你到底要什麼」
2. 五行規格：做什麼／給誰用／輸入／輸出／技術限制
3. 驗收條件要可測：另一個人能不能自己判斷通過或沒通過
4. 凡是有標準答案的數字，自己查好貼給它——它會編
5. 動手之前先讀它的實作計畫，理解錯了在那裡攔下來
6. 五級光譜：依「錯了會怎樣」與「我驗不驗得出來」調強度

下一步　下一週：迭代與除錯——你這一版一定有錯（那是刻意的）。我們會學怎麼讀錯誤訊息、怎麼用證據而不是形容詞跟 AI 溝通，並把三個 bug 全部修好。